



Pontifícia
Universidade
Católica do
Rio de Janeiro

DEI
DEPARTAMENTO
DE ENGENHARIA
INDUSTRIAL



ENG 4560 – Projeto Integrado VI: Distribuição Física | 2026.1

Aula 12: Análise de sensibilidade – ILS

Prof. Marcello Congro

marcellocongro@puc-rio.br



CTC
CENTRO TÉCNICO CIENTÍFICO / PUC-Rio

Onde estamos no curso?

- Na Aula 11, a pergunta era simples: o ILS funciona? A resposta foi sim: o algoritmo melhorou a busca local em todas as instâncias testadas. Mas um resultado positivo não é o fim da análise; é o começo.



A pergunta da Aula 12 é mais exigente: como o ILS se comporta? Por que ele melhora? Quando ele para de melhorar? O que isso diz sobre o problema da Prolog e sobre a configuração escolhida?

- Um engenheiro que só entrega o custo final não consegue defender sua recomendação diante de uma banca, nem diagnosticar o que mudar quando o resultado não for satisfatório.
 - ❖ A diferença entre as duas aulas é a diferença entre usar uma ferramenta e compreender uma ferramenta. A Aula 12 entrega a segunda habilidade.



<https://prologtransportes.com.br/>

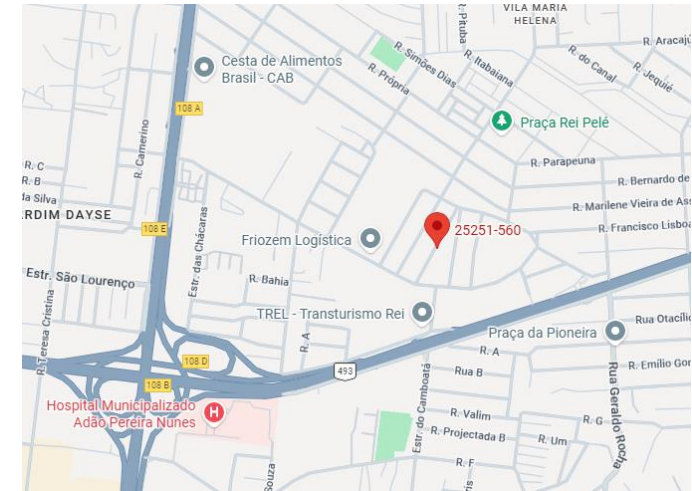
Onde estamos no curso? (cont.)

- O que vamos analisar na aula de hoje?
 - **Tema 1** — Análise de sensibilidade: quanto o resultado muda quando ajustamos k (intensidade da perturbação) e N (iterações)? Existe uma combinação claramente superior? Como identificá-la empiricamente?
 - **Tema 2** — Diagnóstico de convergência: quando o ILS para de melhorar, por quê? Há diferença entre um algoritmo que convergiu genuinamente e um que ficou preso prematuramente. Esse diagnóstico muda completamente a recomendação!
 - **Tema 3** — Escalabilidade: o ILS é mais valioso à medida que as instâncias crescem? O tempo computacional cresce de forma aceitável para a operação da Prolog?
- Os três temas se encadeiam: a sensibilidade diz o que ajustar, o diagnóstico diz quando parar, e a escalabilidade diz onde o ILS é mais valioso.



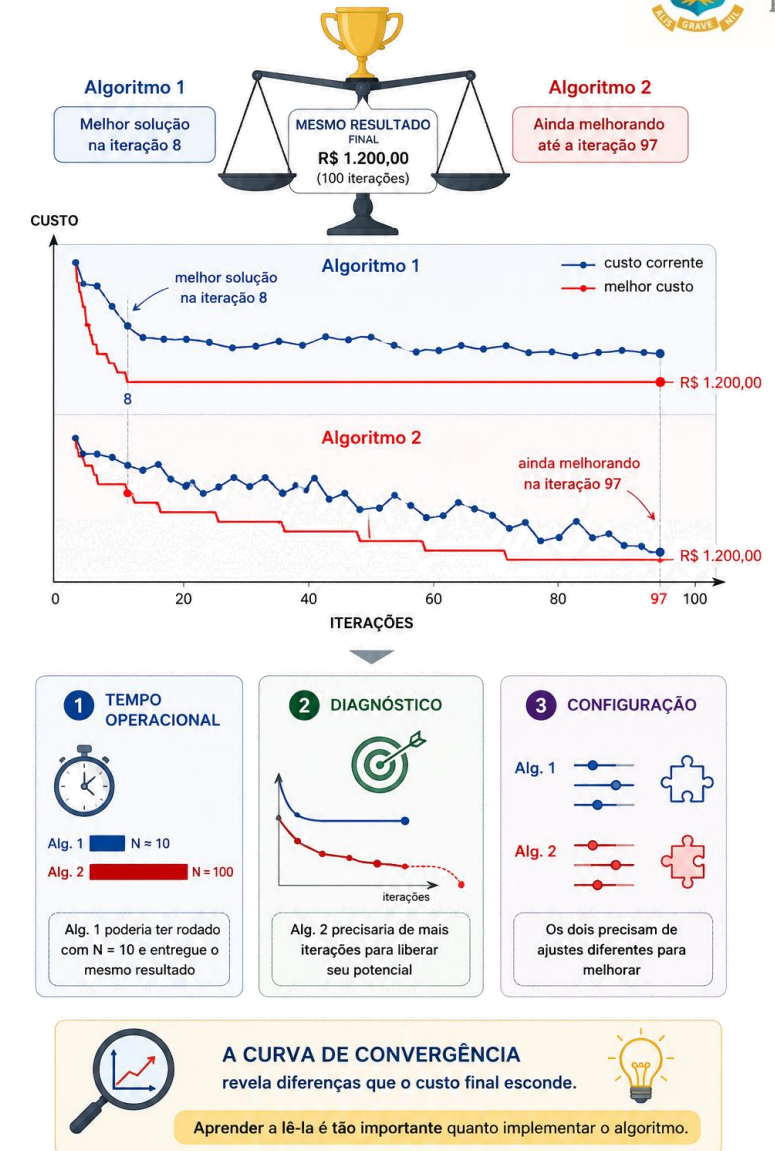
Características do sistema logístico

- O sistema logístico do projeto considerado nesta disciplina será caracterizado pelos seguintes parâmetros reais de operação:
 - ❖ Depósito (Centro de Distribuição – CD): **CEP 25251-560**
 - ✓ Este ponto será o local de origem e retorno de todas as rotas.
 - ❖ Tipos de veículos disponíveis:
 - ✓ Fiorino: Capacidade máxima **650 kg**, custo fixo diário **R\$ 250,00**;
 - ✓ VUC: Capacidade máxima **3000 kg**, custo fixo diário **R\$ 550,00**.
 - ❖ Custos e tempos adicionais:
 - ✓ Custo variável: R\$ 1,50/km (aplicável a todos os veículos);
 - ✓ Velocidade média: 40 km/h;
 - ✓ Tempo de atendimento por cliente: 15 min;
 - ✓ Jornada máxima diária: 8h (= 480 min).
- Esses parâmetros serão utilizados como entrada no problema, e, portanto, devem ser registrados e mantidos consistentes ao longo das análises.



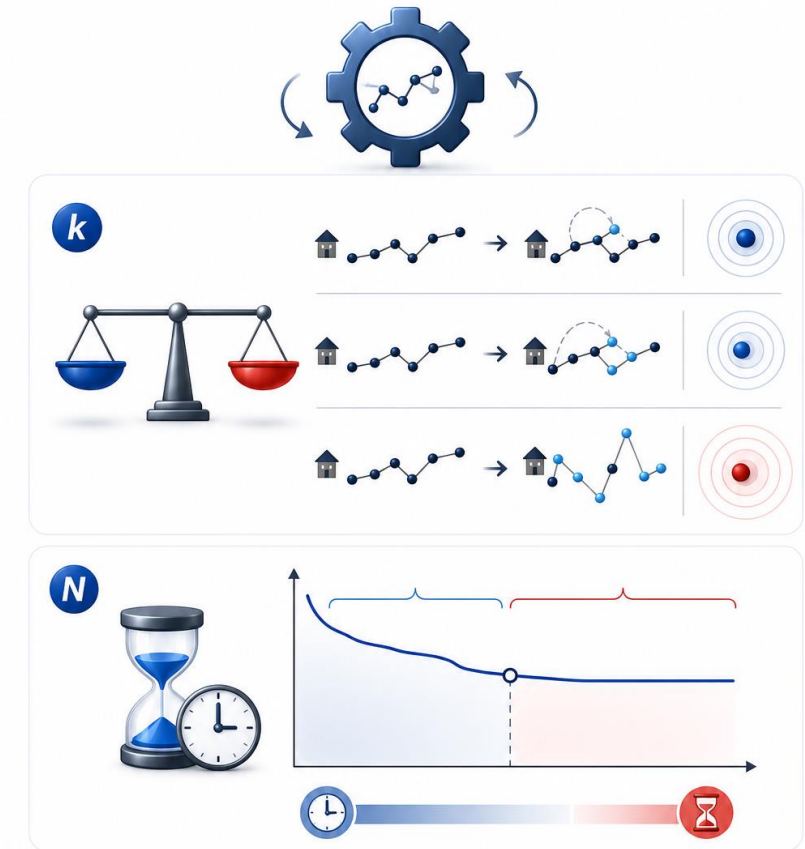
Por que analisar além do custo final?

- O custo final é a métrica mais importante, mas não é a única. Considere dois algoritmos que entregam exatamente R\$1.200,00 ao final de 100 iterações. São equivalentes?
 - ❖ O primeiro encontrou sua melhor solução na iteração 8 e ficou as 92 iterações restantes sem nenhuma melhoria;
 - ❖ O segundo ainda estava melhorando na iteração 97 quando foi interrompido. O comportamento é oposto, mas o número final é o mesmo.
 - ❖ Isso importa de três formas: (1) tempo operacional — o primeiro poderia ter rodado com $N=10$ e entregado o mesmo resultado; (2) diagnóstico — o segundo precisaria de mais iterações para liberar seu potencial; (3) configuração: os dois precisam de ajustes diferentes para melhorar.
 - ❖ A curva de convergência é o instrumento que revela essas diferenças. Aprender a lê-la é tão importante quanto implementar o algoritmo.



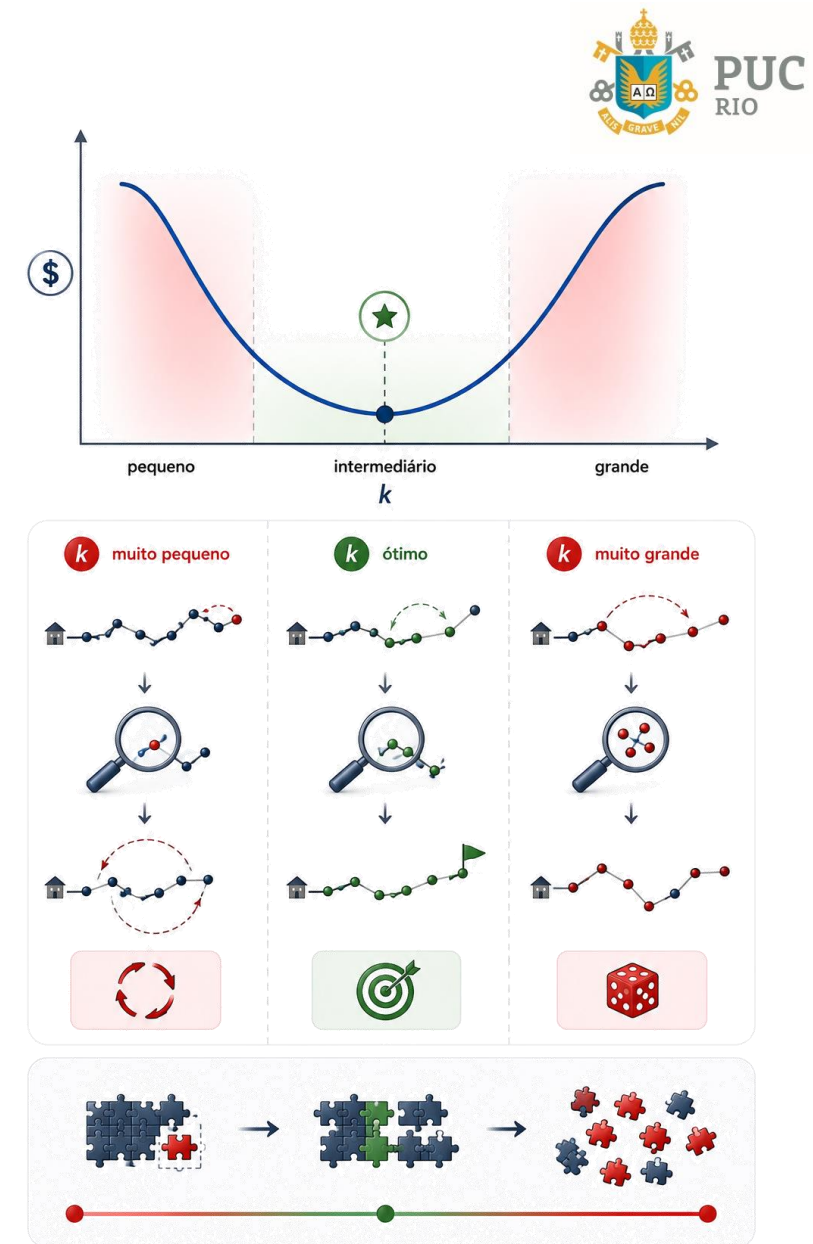
Parâmetros principais do ILS

- O ILS tem um conjunto pequeno de parâmetros:
 - ❖ k — *intensidade da perturbação*: quantos clientes são movidos aleatoriamente a cada iteração. Controla o equilíbrio entre escapar do ótimo local (diversificação) e preservar a estrutura da solução (intensificação). Perturbações muito fracas não escapam; perturbações muito fortes destroem.
 - ❖ N — *número de iterações*: por quanto tempo o algoritmo continua tentando. Controla diretamente o tempo computacional — o único recurso que a Prolog gerenciará na operação real. Mais iterações custam tempo; iterações além do ponto de inflexão custam tempo sem retorno.
 - ❖ A regra de ouro do experimento controlado: mude um parâmetro de cada vez, mantendo tudo mais fixo (seed, instância, solução inicial, critério de aceitação). Se dois parâmetros mudam ao mesmo tempo e o resultado melhora, não sabemos qual foi responsável.



Por que k importa?

- Existe uma teoria clara sobre o comportamento ao variar k : o custo final deve seguir uma **curva em U**: alto para valores pequenos, mínimo em um valor intermediário, alto novamente para valores grandes
- k muito pequeno ($k = 1$): a perturbação é fraca demais. A busca local, aplicada após a perturbação, desfaz o movimento e o algoritmo retorna ao mesmo ótimo local de onde partiu. O ILS fica rodando em círculos sem progredir.
- k muito grande: a perturbação destrói a estrutura da solução. O resultado é equivalente a um reinício aleatório, perde-se toda a informação acumulada nas iterações anteriores. O algoritmo perde memória a cada ciclo.
- k ótimo: existe um ponto de equilíbrio onde a perturbação é suficientemente agressiva para escapar, mas suficientemente controlada para preservar a estrutura. Esse ponto depende da instância. A análise de sensibilidade existe justamente para encontrá-lo empiricamente.



Efeito de k

- O experimento mantém $N=80$ e seed = 42 fixos e varia apenas $k \in \{1, 2, 3, 4\}$. Isso garante que qualquer diferença nos resultados é causada por k , não por acaso ou outras configurações.
- O que observar nas tabelas: **qual valor de k produziu o menor custo final? A diferença entre os valores é grande ou pequena?** Isso indica se k é um parâmetro crítico ou secundário para esta instância.
- O que observar nas curvas de convergência: o formato é diferente para cada k ? k alto converge mais rapidamente (melhora muito nas primeiras iterações) mas estabiliza mais cedo? Essa é a assinatura de uma perturbação agressiva.
- O melhor k encontrado será usado no Experimento 2. Registre-o.



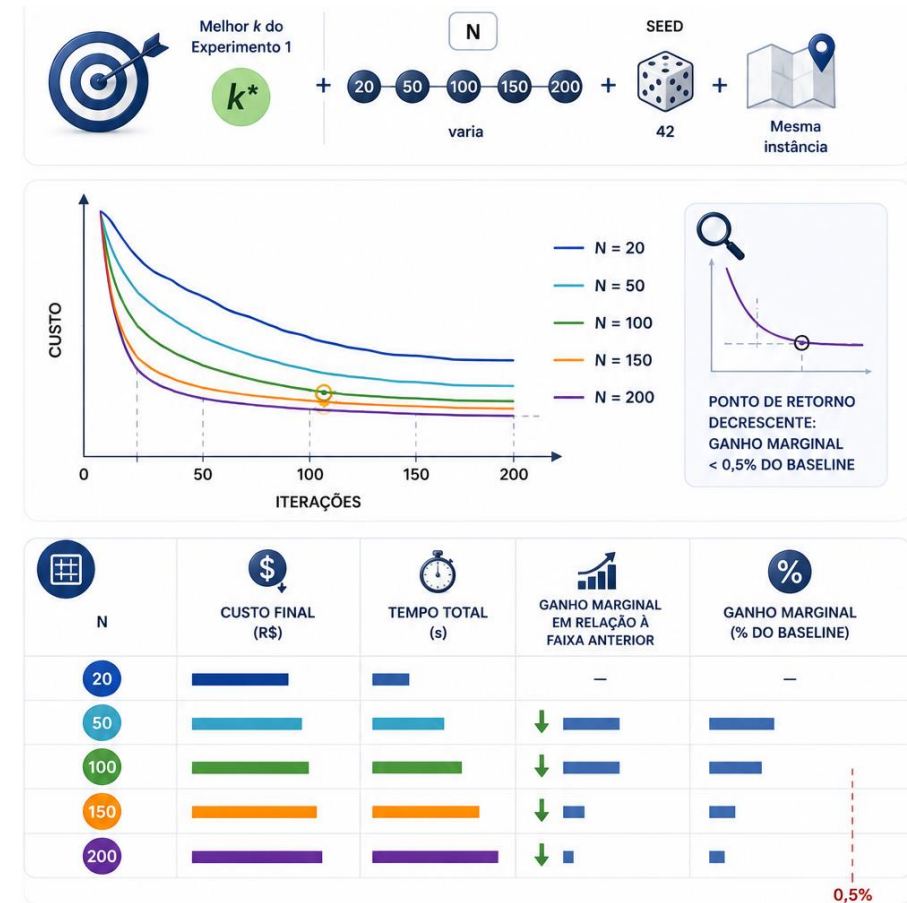
Efeito de N

- Cada iteração adicional do ILS tem custo computacional fixo (sempre leva o mesmo tempo) mas ganho esperado decrescente (as melhorias ficam cada vez menores e mais raras).
- A razão é intuitiva: as primeiras iterações encontram melhorias fáceis (escapar dos piores ótimos locais da solução inicial). As iterações tardias tentam ajustes cada vez mais finos em regiões já exploradas, onde as melhorias são pequenas e raras.
- Isso cria um ponto de inflexão: além de um certo N , cada iteração adicional contribui menos de 0,5% de melhoria sobre o custo baseline. O algoritmo está trabalhando em vão, pois o tempo computacional sobe, mas o resultado quase não muda.
- Identificar esse ponto é a diferença entre um algoritmo útil operacionalmente e um que demora mais do que a janela de planejamento permite. A recomendação final deve especificar o valor de N .



Efeito de N (cont.)

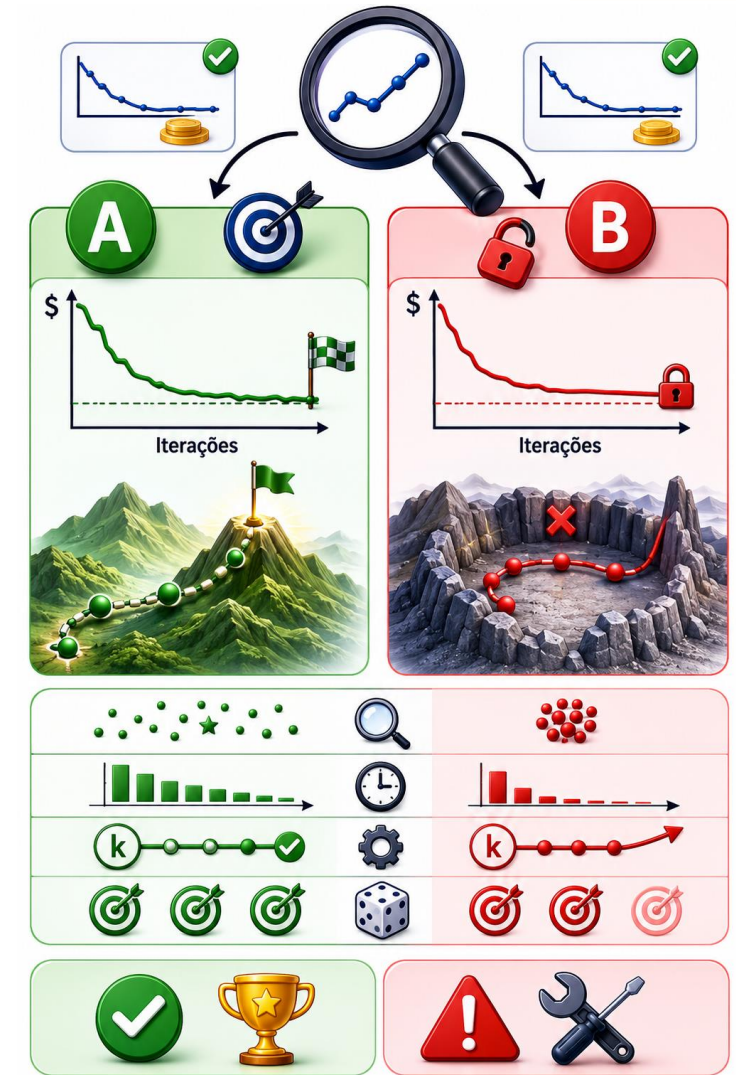
- O experimento usa o melhor k do Experimento 1 e varia $N \in \{20, 50, 100, 150, 200\}$. Mantém seed = 42 e a mesma instância de trabalho.
- Ganho marginal: para cada faixa de N , calcule quanto o custo reduziu em relação à faixa anterior. O custo ainda estava caindo significativamente ao passar de $N=100$ para $N=150$? Ou a diferença foi desprezível?
- Ponto de retorno decrescente: identifique o N a partir do qual o ganho marginal cai abaixo de 0,5% do custo baseline.
 - Esse é o N prático recomendado, mais iterações além desse ponto têm custo computacional sem retorno relevante.
- Se a Prolog tiver 2 minutos para gerar o plano de entregas antes do início da jornada, qual N cabe nessa janela? A resposta combina os dados de tempo da tabela com o ganho marginal.



Existe algum N a partir do qual não houve nenhuma melhoria adicional?

Diagnóstico de convergência

- A análise de sensibilidade respondeu quanto os parâmetros importam. O diagnóstico de convergência responde uma pergunta mais sutil: *por que o algoritmo parou de melhorar?*
- Existem duas razões possíveis, com implicações completamente diferentes:
 - **Razão A** — Convergência genuína: o algoritmo explorou bem o espaço e chegou a um bom ótimo local. Mais iterações com a mesma configuração não ajudariam. O resultado é confiável;
 - **Razão B** — Estagnação prematura: a perturbação não foi forte o suficiente para escapar de um ótimo ruim. Mais iterações com a mesma config não ajudam. É preciso mudar k ou o tipo de perturbação.
- O problema: os dois casos podem ter o mesmo custo final e a mesma curva de convergência aparente



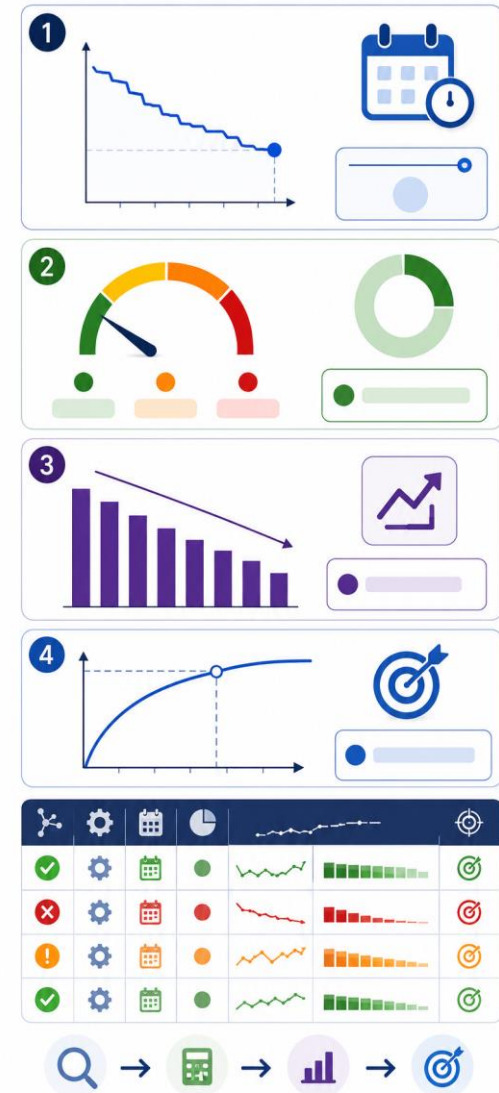
Convergência genuína vs. estagnação prematura

- A distinção entre os dois casos se detecta pela iteração da última melhoria: **em que momento o melhor custo global foi atualizado pela última vez?**
 - Convergência genuína:** a última melhoria aconteceu próximo ao final (ex.: iteração 90 de 100). O algoritmo ainda estava aprendendo quando foi interrompido. A região ociosa é pequena. **Considere aumentar N se houver tempo disponível.**
 - Estagnação prematura:** a última melhoria aconteceu muito cedo (ex.: iteração 12 de 100). O algoritmo ficou 88 iterações sem progredir. Isso não é convergência, é paralisia. A perturbação não foi suficiente para escapar do ótimo local onde o algoritmo ficou preso. **Aumente k ou mude o tipo de perturbação.**
- Percentual ocioso = $(N - \text{última_melhoria}) / N \times 100$.
 - Acima de 50%: sinal claro de estagnação;
 - Abaixo de 20%: algoritmo ainda tinha potencial quando parou.



Métricas de diagnóstico

- **Iteração da última melhoria:** o número mais importante do diagnóstico. Registre para cada instância e configuração. Este é o dado que sustenta a classificação entre convergência genuína e estagnação.
- **Percentual ocioso:** fração do tempo computacional sem nenhum progresso no melhor custo global. Acima de 50% indica estagnação clara; entre 25–50% é convergência parcial; abaixo de 25% é convergência ativa.
- **Taxa de aceitação por bloco de iterações:** fração das perturbações aceita como novo ponto de partida em cada bloco de 10–15 iterações. Uma queda progressiva ao longo do tempo é o sinal mais claro de convergência genuína (o algoritmo está explorando regiões de custo cada vez mais semelhantes).
- **Curva de ganho acumulado:** expressa o ganho percentual sobre o baseline à medida que as iterações avançam. A marcação de onde 90% do ganho total foi capturado identifica o N prático. O número mínimo de iterações para extrair a maior parte do valor do algoritmo.



Strict vs. Tolerance

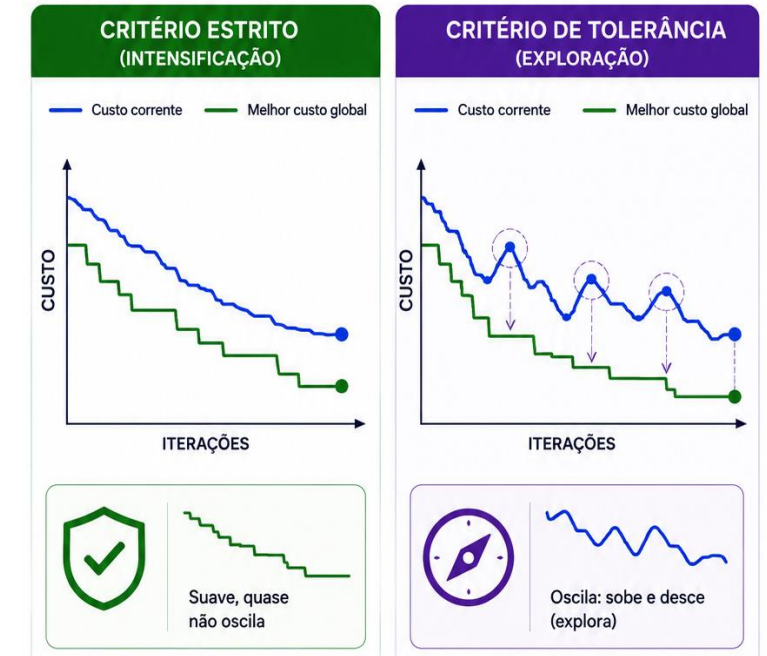
- A comparação entre os dois critérios de aceitação mostra, de forma visível nos gráficos, o trade-off fundamental entre intensificação (estrito) e exploração (tolerância).

➤ **Critério estrito:** a linha do custo corrente (azul) é suave e quase idêntica à linha do melhor custo (nunca sobe). O comportamento é conservador: explora pouco, mas não desperdiça progresso. Pouca c

➤ **Critério de tolerância:** a linha do custo corrente (azul) oscila e sobe e desce. Isso não é defeito: é o mecanismo que aceita temporariamente soluções piores para explorar o espaço de busca.

▪ O que determina qual é melhor?

- QUAL É MELHOR? DEPENDE DA LINHA DO MELHOR CUSTO GLOBAL
- Olhe para a linha do melhor custo global (verde). Se a linha do custo corrente (azul) estiver sempre abaixo ou igual à linha do melhor custo global, o critério estrito é melhor.
 - As duas chegam ao mesmo ponto? A tolerância foi neutra. O melhor com tolerância é pior? A exploração não foi produtiva para essa instância.



O resultado esperado depende do tamanho da instância: em instâncias pequenas (C1, C2), os dois critérios tendem a convergir para resultados similares. Em instâncias maiores (C3, C4), a tolerância tem mais chance de mostrar vantagem

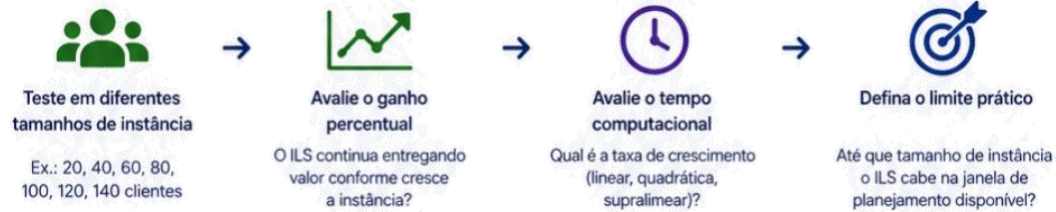
Strict vs. Tolerance

- A comparação entre os dois critérios de aceitação mostra, de forma visível nos gráficos, o trade-off fundamental entre intensificação (estrito) e exploração (tolerância).
 - **Critério estrito:** a linha do custo corrente (azul) é suave e quase idêntica à linha do melhor custo (nunca sobe). O comportamento é conservador: explora pouco, mas não desperdiça progresso. Pouca oscilação visível.
 - **Critério de tolerância:** a linha do custo corrente oscila visivelmente, sobe e desce. Isso não é defeito: é o mecanismo de exploração em ação. O algoritmo aceita temporariamente soluções piores para se posicionar em novas regiões.
- **O que determina qual é melhor?**
 - Olhe para a linha do melhor custo global. Com tolerância ela desce mais? Então a exploração gerou valor.
 - As duas chegam ao mesmo ponto? A tolerância foi neutra. O melhor com tolerância é pior? A exploração não foi produtiva para essa instância.



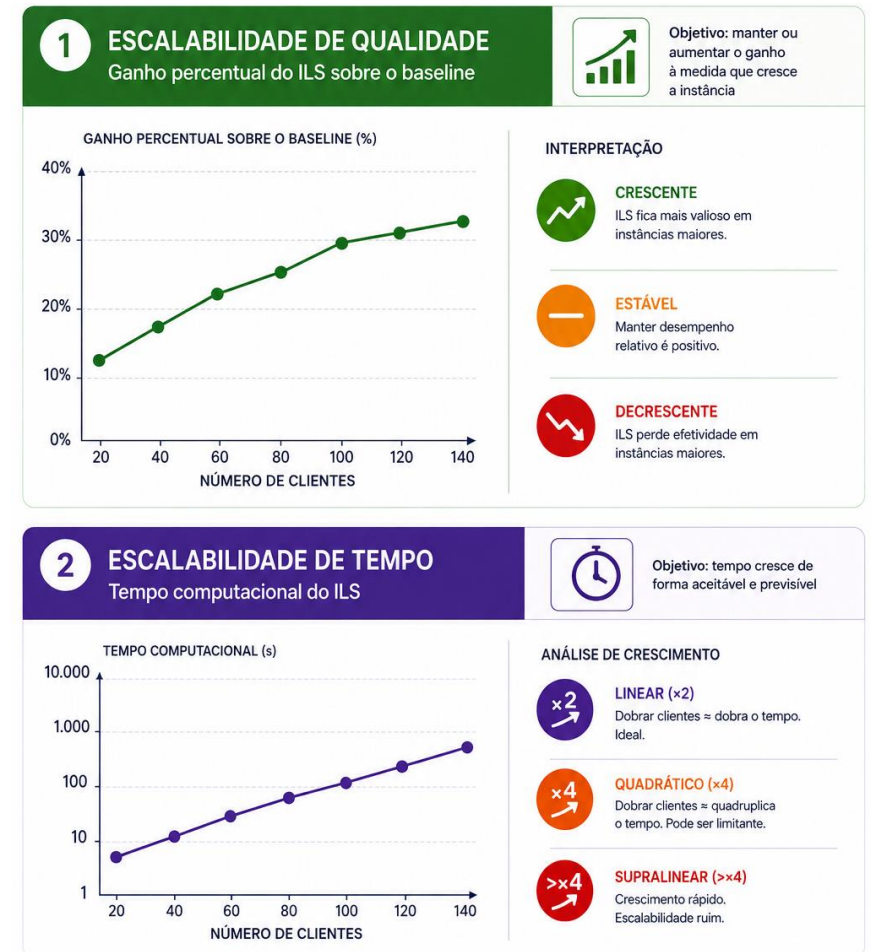
O resultado esperado depende do tamanho da instância: em instâncias pequenas (C1, C2), os dois critérios tendem a convergir para resultados similares. Em instâncias maiores (C3, C4), a tolerância tem mais chance de mostrar vantagem

Escalabilidade



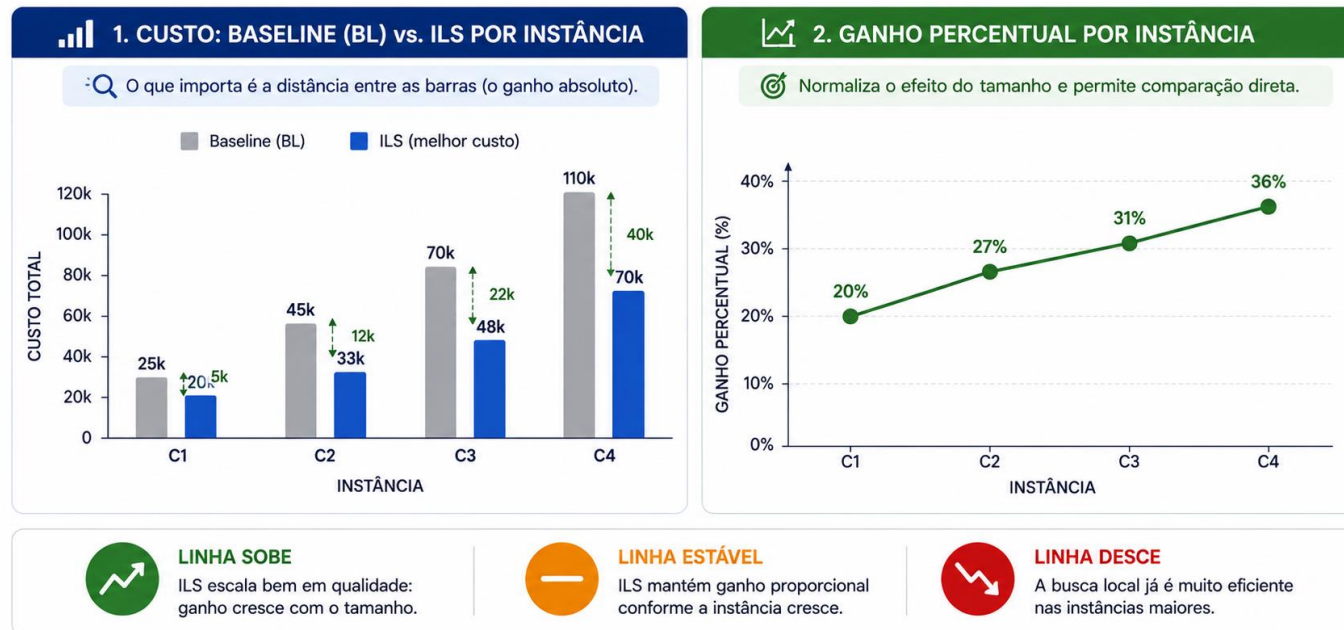
- A escalabilidade pergunta: e quando a instância cresce? A Prolog não tem sempre o mesmo número de clientes por dia: a demanda varia, e o algoritmo precisa funcionar bem em diferentes escalas.
- Escalabilidade tem dois componentes que precisam ser analisados separadamente:
 - (1) **Escalabilidade de qualidade**: o ganho percentual do ILS sobre o baseline cresce, diminui ou é estável com instâncias maiores?
 - (2) **Escalabilidade de tempo**: o tempo computacional cresce de forma aceitável? Se dobrar os clientes quadruplica o tempo, isso precisa estar na recomendação.

Em instâncias pequenas, o espaço de soluções é limitado. A busca local já encontra bons ótimos locais por conta própria, e o ILS acrescenta pouco. Em instâncias grandes, o espaço cresce exponencialmente, os ótimos locais se multiplicam e a perturbação torna-se mais necessária.



Escalabilidade (cont.)

- **Gráfico de barras** (custo BL vs. ILS por instância): o que importa não é o nível absoluto das barras (que cresce naturalmente com a instância), mas a distância entre elas. Ela aumenta de C1 para C4? Se sim, o ILS ganha mais terreno conforme o problema cresce.
- **Gráfico de linha** (ganho percentual por instância): normaliza o efeito do tamanho e permite comparação direta. Se a linha sobe de C1 para C4, o ILS escala bem em qualidade. Se desce, a busca local já era competente nas instâncias maiores e o ILS acrescenta proporcionalmente menos.



Escalabilidade (cont.)



- O notebook gera quatro gráficos de escalabilidade. Cada um conta uma parte diferente da história e deve ser descrito e estudado detalhadamente.
 - **G1 — Custo BL vs. ILS por instância:** a distância entre as barras (baseline cinza e ILS verde) aumenta de C1 para C4? Uma distância crescente confirma a hipótese de escalabilidade de qualidade.
 - **G2 — Ganho percentual por instância:** a linha ascendente é o argumento mais direto. Se sobe: ILS escala bem. Se desce: BL já é competente nas instâncias maiores. Se é plana: ganho consistente mas que não cresce — o ILS é útil em todas as escalas, mas não se beneficia adicionalmente de instâncias maiores.
 - **G3 — Tempo computacional empilhado:** o tempo de ILS cresce proporcionalmente mais do que o tempo de BL em instâncias maiores? Se sim, a relação custo-benefício do ILS piora com o tamanho — isso precisa aparecer na recomendação.
 - **G4 — Convergência normalizada (% do baseline):** ao expressar o custo como porcentagem do baseline de cada instância, todas as curvas ficam na mesma escala. As curvas de C3 e C4 descem mais do que as de C1 e C2? Isso confirma que o ILS é relativamente mais eficaz nas instâncias maiores.